

Stochastic-Volatility State-Space Model: A Full HMC Pipeline, Identifiability, and a Neural Accelerator

Amresh Verma

Abstract

We present a complete estimation pipeline for the (nonlinear, non-Gaussian) stochastic-volatility state-space model (SVSSM): prior specification, a differentiable particle-filter likelihood, Hamiltonian Monte Carlo (HMC) sampling in TensorFlow Probability (TFP), and analysis of the posterior. We then answer, one by one, the questions on JIT/XLA efficiency and the cost of differentiating Sinkhorn, parameter recovery, the initial condition, the modified (additive) observation model, parameter identifiability, the design *and training* of the neural operator used inside the filter (loss, the high-dimensional determinant, and jit-compilation inside HMC), and the use of a Hamiltonian neural network to accelerate sampling. Every claim is backed by an experiment, with all supporting numbers reported in-line so the document is self-contained.

Contents

1	Model and estimation pipeline	2
2	Efficient differentiation: JIT, retracing, and the case for a neural operator	3
3	Recovering the parameters; should θ enter the operator?	5
4	The initial condition h_0	8
5	Estimating A in the additive model	8
6	Restrictions on A: what makes the model “meaningful”	9
7	Which parameters need restricting for identifiability?	10
8	Identification: can different parameters give the same data?	12
9	Design of the neural operator	14
10	Training the operator: loss, the determinant, and JIT inside HMC	16
11	Accelerating HMC with a Hamiltonian neural network (L-HNN)	16

1 Model and estimation pipeline

The model (V1). The latent log-volatility h_t follows a stationary AR(1) and is observed multiplicatively. In one dimension,

$$h_t = \mu + \phi(h_{t-1} - \mu) + \sigma_\eta \eta_t, \quad \eta_t \sim \mathcal{N}(0, 1), \quad |\phi| < 1, \quad (1)$$

$$y_t = \exp(h_t/2) \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, 1). \quad (2)$$

The parameters are $\theta = (\mu, \phi, \sigma_\eta)$. In d dimensions $h_t \in \mathbb{R}^d$, with $\mu \in \mathbb{R}^d$, a (here upper-triangular) persistence matrix Φ , a diagonal shock covariance Σ_η , and $y_{t,i} = \exp(h_{t,i}/2) \varepsilon_{t,i}$ component-wise. Stationarity requires the eigenvalues of Φ to lie inside the unit disc.

The modified observation (V2). Here the same AR(1) state (1) is taken, however in the multivariate case we drop the cross-asset persistence, and the observation is replaced by

$$y_t = A h_t + \sigma_\varepsilon \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, I), \quad (3)$$

where A is a scalar (1D) or a $d_y \times d_h$ matrix (multivariate).

The pipeline. Estimation is fully Bayesian and proceeds in four stages.

1. **Prior.** We work in an unconstrained parameterisation so HMC sees $\mathbb{R}^{\dim \theta}$: $\phi = \tanh(\phi_{\text{raw}})$, $\sigma_\eta^2 = \exp(\ell)$, μ free. Independent Gaussian priors are placed on $(\mu, \phi_{\text{raw}}, \ell)$; their roles (and when they are *needed* vs. merely helpful) are taken up in the identifiability section.
2. **Likelihood.** The observation is nonlinear/non-Gaussian, so $p(y_{1:T} | \theta)$ has no closed form. We estimate $\log p(y_{1:T} | \theta)$ with a *differentiable particle filter* (DPF): a localized exponential-integrator particle-flow (LEDH) filter whose resampling step is an optimal-transport (OT) map. The map is computed either by Sinkhorn (exact, $O(N^2 K)$) or by a trained neural operator [4] (NN-OT, $O(N)$). Crucially the whole filter is differentiable in θ , so $\nabla_\theta \log p(y_{1:T} | \theta)$ is available by automatic differentiation, this is precisely what lets the DPF *support* HMC.
3. **HMC.** We sample the posterior $p(\theta | y_{1:T}) \propto p(y_{1:T} | \theta) p(\theta)$ with TFP’s Hamiltonian Monte Carlo. The production kernel is a windowed-adaptive, dense-mass (full inverse-covariance) preconditioned HMC. Common random numbers (a fixed filter seed) are shared across chains so the stochastic likelihood surface is identical chain-to-chain. Two samplers are used *interchangeably* on the same posterior: (i) this exact-gradient kernel, and (ii) an L-HNN NUTS [5] sampler (§11) that swaps the expensive filter gradient for a trained Hamiltonian-neural-network surrogate and corrects against the real target with a Metropolis step. The exact kernel is preferred where the gradient is affordable (low T , where it also mixes better); the surrogate wins at high T , where each exact gradient is costly (the crossover is measured in §11).
4. **Analysis.** Convergence and recovery are assessed with rank-normalised split- $\hat{R}$, bulk/tail effective sample size (Vehtari et al. 2021) [1], 95% credible-interval coverage of the truth, trace plots, and posterior-vs-prior shrinkage to separate data-driven identification from prior dominance.

The observation likelihood: HRS quasi-likelihood and its structural bias. The multiplicative observation (2) is handled by the standard SV linearisation, i.e., square and take logs,

$$z_t := \log(y_t^2 + \delta) = h_t + \mu_z + e_t, \quad (4)$$

with a small offset δ to keep the log finite. The true noise $\log \varepsilon_t^2$ is $\log \chi_1^2$: left-skewed, with mean $\mu_z = -1.2704$ and variance $\pi^2/2 \approx 4.93$. The Harvey–Ruiz–Shephard (HRS, 1994) [2] *quasi*-likelihood replaces this skewed noise by a single Gaussian $e_t \sim \mathcal{N}(0, 4.93)$, matching only its first two moments. This is the key modelling assumption in our likelihood, and a deliberate trade: the single Gaussian is smooth and differentiable in θ (required for HMC, and for the neural operator of §9), whereas the exact $\log \chi_1^2$ or the sharper Kim–Shephard–Chib (KSC, 1998) [3] seven-component Gaussian mixture would complicate the differentiable filter and is deferred.

The assumption is not free. Matching only the first two moments of a highly skewed observation distribution induces a *structural* misspecification of the observation likelihood. This misspecification can bias the inferred state dynamics and often manifests as overestimation of the volatility shock variance σ_η^2 , and consequently of the stationary variance $\sigma_h^2 = \sigma_\eta^2/(1 - \phi^2)$. We flag it here because it recurs in the results and must not be read as a sampler or Monte-Carlo artefact: it does *not* vanish with more data (the σ_h^2 posterior *loses* 95% coverage as T grows, even at full convergence with rank- $\hat{R} \leq 1.07$; §8), and it does not diminish as the particle count N grows, ruling out a finite-particle artefact and locating the bias in the (HRS) likelihood itself. This forces a clean distinction we return to in §8: *identifiability* (does the likelihood pin a parameter as data grows?) is separate from this *estimation bias* (does the approximate likelihood centre on the truth?). In our results σ_h^2 is *identified-but-biased*, its posterior concentrates with T , but around a value shifted by the HRS approximation.

Roadmap. §2 covers efficient differentiation (JIT, retracing, the cost of differentiating Sinkhorn, and why a neural operator is needed); Section 3 estimates the parameters back and asks whether θ should enter the operator; §4 treats the initial condition h_0 ; §5–§7 treat the additive model (3) (A , “meaningfulness”, and which parameters need restricting); §8 is the identification analysis; §9 is the neural-operator design (differentiability, smoothness, monotonicity); §10 is its training (loss, the determinant problem, and JIT inside HMC); and §11 is the optional Hamiltonian-neural-network accelerator.

2 Efficient differentiation: JIT, retracing, and the case for a neural operator

Question. HMC needs $\nabla_\theta \log p$ thousands of times; without XLA/JIT the filter is far too slow. How do we guarantee no retracing and a jit-compilable LEDH-PF-PF-OT? Can pure Sinkhorn be differentiated by autodiff; how costly, and is it practical? Why a neural operator?

All numbers below are for the project’s actual HMC target, the SVSSM particle-filter likelihood `DifferentiableLEDHLogLikelihoodSVSSM`, measured on CPU (TF 2.16.2) at $T=20, N=64, n_\lambda=10$, Sinkhorn $K=10$, exp-integrator.

No retracing, and why it matters. A `@tf.function` retraces whenever it sees a new input signature or a *Python*-valued argument that changes and HMC changes θ at every proposal. If θ were passed as a Python float the filter would retrace every leapfrog step (each trace costs seconds), making HMC unusable. We pass θ as a `tf.Tensor` and keep all control flow tensor-based (a `tf.while_loop` over time), so each jitted kernel traces *once*. Verified on this filter: across

forward, gradient, and full-HMC passes the post-warm-up trace-count increment is **0** for every kernel (`_timestep_1d_xla`, `sinkhorn_potentials`), for all three h_0 init types, with finite gradients on all 10 distinct- θ evaluations.

JIT is not optional, it gives 50-100 $\times$ speedup. The gradient is what HMC consumes:

	cold (first call: trace+compile)	warm (jitted)	speedup
forward	1.69 s	14.7 ms	115 $\times$
forward + grad	5.48 s	105 ms	52 $\times$

The cold cost (trace+XLA compile) is paid once; the warm gradient at 105ms is what makes thousands of leapfrog steps feasible. Un-jitted, a single gradient is ~ 5 s that means a 2500-step chain would take hours *per gradient sweep*. The backward pass is $7.1\times$ the forward (the LEDH flow and the unrolled Sinkhorn iterations are re-traversed in reverse).

Can pure Sinkhorn be differentiated? Yes. Entropic Sinkhorn is smooth, so autodiff through its unrolled iterations is well-defined and we use it as the exact reference, and the whole filter (Sinkhorn included) is jit-compilable and finite-gradient as shown above. The standalone resample is $O(N^2K)$ with the backward unrolling every iteration:

N	Sinkhorn resample alone		full SVSSM filter	
	fwd (ms)	fwd+grad (ms)	fwd (ms)	fwd+grad (ms)
64	0.50	4.50	14.7	105
128	1.39	6.99	30.5	125
256	2.70	11.2	53.7	200
512	6.40	23.8	127.5	381

Sinkhorn’s forward grows $\sim 13\times$ as N goes $64 \rightarrow 512$ ($8\times$) which is super-linear and consistent with $O(N^2)$. *The honest nuance:* at these N the resample is only 3–5% of the filter’s cost (0.50 of 14.7 ms at $N=64$; 6.4 of 127.5 at $N=512$), *the LEDH particle-flow dominates*, so pure Sinkhorn is entirely practical here. Its $O(N^2K)$ cost (and unrolled backward) only becomes the bottleneck at *large* N .

Why a neural operator, then. Two reasons, both honest about the scale. (i) *Asymptotic scalability:* the operator evaluates the OT map in $O(N)$ with a *fixed-depth* backward (one pass through a small network, no unrolled iterations), so as N grows it replaces the super-linear, iteration-unrolling Sinkhorn with a linear, constant-depth map, which is exactly why the measured NN-OT speedup over Sinkhorn is N -driven (negligible at $N=64$, growing with N). (ii) *Amortisation:* trained once, the operator is reused at every θ the chain visits (it takes θ as input, §3; trained in §10). It is not needed for correctness, Sinkhorn is correct and cheap at small N , but for *scalable, jit-friendly gradients* at the N where the resample would otherwise dominate.

Profiling (efficiency with HMC). A full HMC step (5-leapfrog, the same filter) runs at a warm median of 0.64 s after a one-off ~ 0.8 s first step; the cost is dominated by the (already XLA-fused) filter recursion, 5 gradient evaluations per step at ~ 105 ms each, with no Python-side hot spots, consistent with the zero-retracing result. This is the per-step budget that the L-HNN accelerator of §11 attacks by replacing the filter gradient with a surrogate.

The multivariate filter profiles near-identically ($d=2$). The diagonal- Φ multivariate filter (...SVSSMmulti, the $d=2$ HMC target) shows the same Section-1 behaviour: zero retracing (_jit_step trace-delta 0 over 10 evaluations), a JIT speedup of 49–106 $\times$ (cold forward 1.75 s $\rightarrow$ warm 16.6 ms; cold fwd+grad 5.36 s $\rightarrow$ warm 109.5 ms, backward 6.6 $\times$), and a warm HMC step of 0.65 s. These sit within a few percent of the univariate figures (105 ms gradient, 0.64 s step), going to $d=2$ adds *negligible* cost, because XLA fuses the per-component matrix operations into the same recursion. Full-filter forward grows 17 $\rightarrow$ 190 ms over $N=64 \rightarrow 512$, the same super-linear shape as $d=1$.

3 Recovering the parameters; should θ enter the operator?

Question. Given data generated at $(\mu^*, \phi^*, \sigma_\eta^*)$, recover the parameters with HMC in TFP; first in 1D, then in higher dimensions. Should the model parameters be part of the neural operator’s input?

Recovery in 1D (data-driven, under a wide prior). The test uses a *wide, shifted* prior whose modes are all far from the truth, so any recovery is driven by the data and not by prior agreement: $\mu \sim \mathcal{N}(2, 3^2)$ (mode 2, truth 0), $\phi_{\text{raw}} \sim \mathcal{N}(0, 2^2)$ (mode $\phi=0$, truth 0.95), $\log \sigma_\eta^2 \sim \mathcal{N}(1.5, 3^2)$ (mode $\sigma_\eta^2=4.48$, truth 0.09). We sweep $T \in \{50, 100, 200\}$ with 4 chains $\times$ 2500 draws each; rank- $\hat{R} \leq 1.044$ throughout.

param	truth	med ($T=50$)	med ($T=100$)	med ($T=200$)	95% CI ($T=200$)
μ	0.00	0.70	0.36	0.23	[−2.05, 1.93]
ϕ	0.95	0.87	0.92	0.954	[−0.70, 0.998]
σ_η^2	0.09	0.195	0.286	0.101	[0.004, 0.982]

All parameters are covered at every T , but the three carry *different* evidentiary weight, and it is worth being precise about what the data (rather than the prior) actually does for each. We calibrate against a *prior-only* run (likelihood switched off, so the target is the pure prior; --no_likelihood), which shows the induced prior in the constrained coordinates.

μ and σ_η^2 are genuinely data-driven from off-truth priors. The level μ migrates from its prior mode 2.0 toward the truth (0.70 $\rightarrow$ 0.36 $\rightarrow$ 0.23), covering $\approx 88\%$ of the mode-to-truth gap by $T=200$; the residual is μ ’s weak identification compounded with the HRS-on-level floor of §1. σ_η^2 is the cleanest exhibit: its induced prior is *monstrously* heavy-tailed (prior-only median 4.3, with $\approx 38\%$ of draws exceeding $\sigma_\eta^2=10$ and a 99th percentile near 4.7×10^3 , because $\sigma_\eta^2 = \exp(\cdot)$ of a wide log-normal), yet the posterior crushes it to 0.101 (within 0.011 of truth) with a tight CI; the likelihood, not the prior, pins it.

ϕ is more subtle, and the honest reading is that its point value is partly the prior’s doing. Pushing $\phi_{\text{raw}} \sim \mathcal{N}(0, 2^2)$ through tanh does *not* give a prior peaked at $\phi=0$; it gives a *symmetric, edge-piling* density, mass diverging like $1/(1 - \phi^2)$ at ± 1 , with a *trough* at 0 (prior-only $P(\phi < 0) = 49.5\%$, median 0.02). So calling $\phi=0$ the “prior mode” is misleading: the prior already places heavy mass right where the truth (0.95) sits, just inside +1. The point estimate landing at 0.954 therefore rides partly on this boundary edge-piling, *not* on the data alone. What the data genuinely contribute is **symmetry-breaking and concentration**: the prior is agnostic about sign ($P(\phi < 0) = 49.5\%$), and the likelihood drives that down monotonically, 13.6% $\rightarrow$ 8.0% $\rightarrow$ 4.9% across $T = 50, 100, 200$, while the credible interval tightens. So for ϕ the defensible evidence of identification is the collapse of $P(\phi < 0)$ and the shrinking CI *width*, not the median sitting on

truth. (At small T this is visible as a residual *anti*-persistence sub-mode near $\phi \approx -1$, the un-killed lobe, which is why even the $T=200$ interval keeps a negative lower tail.) This is a concrete instance of an identification feature, here the *sign* of persistence, being pinned by more observations; the general theme is §8.

Recovery in $d = 2$, via the L-HNN+NN-OT pipeline. The same recovery succeeds in the multivariate model, here run through the *accelerated* pipeline (NN-OT resampling + L-HNN NUTS; §11) demonstrating that the interchangeable samplers agree. Truth $\mu = (1, -1)$, $\phi_{\text{diag}} = (0.95, 0.80)$, $\phi_{\text{off}} = 0.05$, $\sigma_\eta = (0.312, 0.6)$ (a matched-SNR design with stationary variance $\sigma_h^2 = 1$ per component); priors $\mu \sim \mathcal{N}(0, 1.5^2)$, $\phi_{\text{raw}} \sim \mathcal{N}(1.5, 1)$, $\log \sigma_\eta^2 \sim \mathcal{N}(-1.7, 1)$; $N = 64$, 4 chains $\times 3000$, swept over $T \in \{50, 200, 500, 1000\}$. Posterior medians:

param	truth	$T=50$		$T=200$	$T=500$	$T=1000$	
		med	95% CI	med	med	med	95% CI
μ_0	+1.00	0.41	[−1.44, 2.28]	0.58	0.92	0.94	[−1.25, 2.29]
μ_1	−1.00	−0.51	[−1.63, 0.92]	−0.79	−0.68	−0.87	[−1.58, −0.10]
ϕ_{00}	+0.95	0.91	[0.49, 0.99]	0.94	0.93	0.96	[0.88, 0.99]
ϕ_{11}	+0.80	0.84	[0.07, 0.97]	0.89	0.92	0.93*	[0.84, 0.98]
ϕ_{01}	+0.05	−0.00	[−0.28, 0.31]	0.01	0.01	0.02	[−0.00, 0.07]
$\sigma_{\eta,0}^2$	+0.098	0.19	[0.03, 0.75]	0.23	0.24	0.15	[0.02, 0.37]
$\sigma_{\eta,1}^2$	+0.36	0.18	[0.04, 0.79]	0.25	0.21	0.18	[0.06, 0.38]

Coverage: 7/7 at $T=50, 200, 500$; 6/7 at $T=1000$ (* ϕ_{11} : CI [0.84, 0.98] excludes truth 0.80; the tightened interval exposing the HRS ridge bias). Max rank- $\hat{R}$ across the sweep: 1.02, 1.07, 1.06, 1.16. The CI columns make the sharpening explicit: e.g. ϕ_{00} tightens from [0.49, 0.99] at $T=50$ to [0.88, 0.99] at $T=1000$, and ϕ_{01} from [−0.28, 0.31] to [−0.00, 0.07].

Here the priors are the *moderate* near-truth ones (see the next paragraph), so the evidentiary split differs from the 1D wide case. The level μ (prior mode 0, truth ± 1) genuinely concentrates from an off-truth mode, data-driven. The persistence ϕ_{00} , by contrast, sits at a prior mode of $\tanh(1.5) \approx 0.905$, already close to the truth 0.95, so its concentration is *prior-assisted*, not a clean data-driven recovery (the same edge-piling caution as 1D, now with the raw prior centred on the positive side); the honest multivariate data-driven win is instead the *stationary combinations* of §8 (notably the cross-asset correlation ρ_h). The cross-asset spillover ϕ_{01} recovers only slowly toward 0.05 (weakly identified at these T). The $(\phi_{11}, \sigma_\eta^2)$ coordinates lie on the (ϕ, σ) ridge and carry the HRS variance bias of §1 (ϕ_{11} biased high, σ_η^2 compensating) which is precisely why at $T=1000$ ϕ_{11} ’s tightened interval finally excludes the truth (coverage 6/7): tighter precision exposing a structural bias, not a sampler defect. As in 1D, μ is only *weakly* identified (the level of a log-volatility process carries little information per observation). The full identification structure, the well-identified *stationary combinations* versus the weakly/biasedly identified individual coordinates, is analysed in §8.

Why the accelerated pipeline is clean here, and when it is not. The moderate, near-truth priors above (ϕ prior mode 0.905, σ_η^2 mode 0.18) keep the posterior inside the region where the NN-OT operator and the L-HNN surrogate were trained, which is why the accelerated pipeline is fast and accurate at $d = 2$. This is a *prior-coverage*, not a dimension, property. With a *wide-shifted* prior (the 1D setup above) the posterior is dragged into the tails, where the truth-centred operator grid ($\phi = 0.95 \pm 0.05$) and the single-anchor surrogate are out-of-distribution: the L-HNN H-error monitor then fires constantly and the chain falls back to the real gradient (≈ 8.5 real

gradients per iteration in a $d=1$ test, vs. ≈ 0 here), erasing the speedup. The operational rule is therefore the same one stated in §1: the exact-gradient windowed kernel for wide-prior recovery, the L-HNN+NN-OT pipeline when the prior keeps the posterior inside the trained region (§11).

Higher dimensions. The particle filter, the Φ -matrix exponential propagator, and the HMC driver are implemented dimension-generically: a fixed-order Padé scaling-and-squaring matrix exponential keeps every step XLA-compilable at any d , so the same pipeline runs unchanged at $d = 3, 4, \dots$ (verified numerically against `tf.linalg.exp` to $\sim 10^{-14}$ in float64). The binding constraint at large d is *cost and mixing*, not correctness: exact-gradient HMC scales linearly in T , and the $3d + d(d-1)/2$ -dimensional posterior is harder to traverse — both motivate the accelerator of §11.

Should θ be an operator input? Yes, it is the conditioning that makes the operator usable inside HMC. The neural operator replaces the filter’s optimal-transport (OT) resampling *map*, which transports the weighted particle cloud to an equally-weighted one. That map depends on the current particles and weights, which depend on θ ; and HMC evaluates the filter, hence the map, at *every* θ the chain visits. A θ -agnostic operator would be correct for one parameter and wrong everywhere else, useless for sampling. So θ must enter the operator.

How θ enters — the context vector. We use a DeepONet (branch–trunk) operator (Lu et al. 2021) [6]: a *trunk* network embeds the query location (a particle’s position) into a set of basis functions, a *branch* network maps a *context vector* to the coefficients on that basis, and their inner product gives the transported position. All conditioning lives in the branch’s context vector, of dimension $\text{ctx}(d) = 3d + \frac{d(d-1)}{2} + d + 3$ (7 at $d=1$, 12 at $d=2$), laid out as

$$\left[\underbrace{\mu, \phi_{\text{diag,raw}}, \phi_{\text{off}}, \log \sigma_{\eta}^2}_{\theta \text{ (unconstrained; } 3d + \frac{d(d-1)}{2} \text{ entries)}} ; \underbrace{t/T, z_t, \text{ESS}, \varepsilon}_{\text{per-step filter state (} d+3 \text{)}} \right].$$

The first block is *exactly* the unconstrained parameter vector θ that HMC samples (mean, persistence diagonal in $\tanh^{-1}$ coordinates, off-diagonals, and log shock variances); the second is the per-step filter state, e.g., normalised time t/T , the current log-observation z_t , the effective sample size, and the flow regulariser ε , so the branch produces the right map for the right parameters at the right step. Conditioning on θ this way turns a single learned map into an *operator* over the whole parameter space the chain explores.

Does it work? We test it with a *controlled* comparison: two $d=1$ runs identical in every setting, i.e., same truth, prior, data and common random numbers (`data_seed`, `base_seed`), same L-HNN NUTS sampler, and differing *only* in the resample (trained NN-OT operator vs. exact Sinkhorn), 4 chains $\times$ 1000 each. The full posteriors:

param	truth	NN-OT med [95% CI]	Sinkhorn med [95% CI]	$ \Delta_{\text{med}} $	KS p	Wass.
μ	0.00	−0.31 [−1.08, 0.68]	−0.29 [−1.03, 0.68]	0.018	0.16	0.029
ϕ	0.95	0.805 [−0.34, 0.98]	0.796 [−0.37, 0.97]	0.009	0.0016	0.023
σ_{η}^2	0.09	0.122 [0.03, 0.47]	0.131 [0.03, 0.52]	0.009	0.0017	0.007

Two readings, and both matter. (i) **Point inference is identical:** medians agree to $|\Delta| \leq 0.018$, sd ratios 0.95–1.06, every CI overlapping, at the level any analysis would report, $\phi \approx 0.80$, $\sigma_{\eta}^2 \approx 0.12$, $\mu \approx -0.30$ *whichever* OT is used. (ii) **The distributions are not bit-identical:** with 4000 draws the KS test resolves a tiny shape difference in ϕ and σ_{η}^2 ($p \approx 0.002$), but the effect size is negligible (KS statistic 0.042, Wasserstein ≤ 0.023), the small operator-approximation signature, detectable

but far below anything that changes a conclusion. So the operator is *faithful*: it reproduces the exact posterior up to a sub-0.05 shape perturbation. At $d=2$ the same holds, all 7/7 truths covered with widths within a few percent of Sinkhorn.

4 The initial condition h_0

Question. Estimating the model requires an assumption on the initial latent state h_0 . What should it be, and how does it affect the recovery of §3?

The principled choice: the stationary distribution. Because $|\phi| < 1$, the latent AR(1) (1) has a unique stationary (ergodic) distribution $h_t \sim \mathcal{N}(\mu, \sigma_h^2)$ with $\sigma_h^2 = \sigma_\eta^2 / (1 - \phi^2)$ (in d dimensions Σ_h solves the discrete Lyapunov equation $\Sigma_h = \Phi \Sigma_h \Phi^\top + \Sigma_\eta$). Initialising from it,

$$h_0 \sim \mathcal{N}(\mu, \sigma_\eta^2 / (1 - \phi^2)), \quad (5)$$

is *model-consistent*: h_0 becomes a function of the very parameters (μ, ϕ, σ_η) we estimate, so the earliest observations carry information about them. We compare it against two alternatives: **fixed_mu** ($h_0 \sim \mathcal{N}(\mu, \sigma_\eta^2)$ correct mean but the one-step, not stationary, variance) and **diffuse** ($h_0 \sim \mathcal{N}(0, 100)$ “ h_0 unknown”, deliberately decoupled from the parameters).

Effect on estimation. Ablation at $T=50$ (truth $\mu=0, \phi=0.95, \sigma_\eta=0.3$; 4 chains; deliberately wide priors $\mu \sim \mathcal{N}(0, 5^2)$, $\phi_{\text{raw}} \sim \mathcal{N}(0, 2^2)$, $\log \sigma_\eta^2 \sim \mathcal{N}(-2, 2^2)$). Cell = posterior median [covers truth?]:

h_0 init	h_0 distribution	μ (0)	ϕ (0.95)	σ_η^2 (0.09)
stationary	$\mathcal{N}(\mu, \sigma_\eta^2 / (1 - \phi^2))$	0.13 [Y]	0.947 [Y]	0.133 [Y]
fixed_mu	$\mathcal{N}(\mu, \sigma_\eta^2)$	-0.51 [Y]	0.984 [Y]	0.098 [Y]
diffuse	$\mathcal{N}(0, 100)$	0.96 [N]	0.485 [N]	0.180 [Y]

The choice is *decisive* at short T . The stationary init recovers all three parameters (ϕ median 0.947, truth inside every CI). The **diffuse init fails outright**: μ is biased to 0.96 (CI [0.21, 1.68], excludes 0) and ϕ collapses to 0.485 (CI [-0.70, 0.82], excludes 0.95) both with the truth *outside* the 95% interval. Tellingly, the diffuse chains have the *best* rank- $\hat{R}$ (≤ 1.05 , vs ≈ 1.2 for stationary): they converge confidently to the *wrong* answer. **fixed_mu** is intermediate (covered, but μ pulled to -0.51).

Why diffuse confounds μ and ϕ . At small T the initial transient is a large share of the data. A diffuse h_0 (mean $0 \neq \mu$, variance 100) is decoupled from the parameters, so the early observations no longer pin the dynamics: the filter can absorb them with a wrong low-persistence / high-level combination, biasing ϕ down and μ up. The stationary init removes that freedom because h_0 is tied to (μ, ϕ, σ_η) , so the first observation is informative and the confounding disappears. The effect shrinks as T grows (the init becomes a smaller share of the likelihood), but at the short T of interest it is the difference between recovery and failure. We therefore use the stationary init throughout.

5 Estimating A in the additive model

Question. With the additive observation $y_t = Ah_t + \sigma_\varepsilon \varepsilon_t$ (3) (state unchanged), are there difficulties estimating A ? One dimension first, then A a matrix.

1D: a scale non-identifiability. Scale the latent by any $c > 0$, $\tilde{h}_t = c h_t$: this is again an SVSSM, with $(\tilde{\mu}, \tilde{\sigma}_\eta) = (c\mu, c\sigma_\eta)$, and $y_t = A h_t + \sigma_\varepsilon \varepsilon_t = (A/c) \tilde{h}_t + \sigma_\varepsilon \varepsilon_t$. Hence

$$(\mu, \sigma_\eta, A) \mapsto (c\mu, c\sigma_\eta, A/c), \quad c > 0, \quad (6)$$

leaves the data distribution *exactly* unchanged. So A is **not identifiable on its own**, there is a one-parameter ridge. The data fixes only ϕ , σ_ε , and the scale-invariant products $A\mu$ and $A^2\sigma_\eta^2$ (equivalently $A^2\sigma_h^2$, the variance scale of y): 4 identified quantities for 5 parameters, one free direction.

Demonstrated (1D). HMC on data from $(\mu, \phi, \sigma_\eta, A, \sigma_\varepsilon) = (1, 0.9, 0.5, 2, 0.3)$, $T=200$, with A free vs. A fixed to 1:

param	sd (free A)	sd ($A=1$)	ratio	role
μ	1.79	0.41	4.3×	on ridge
ϕ	0.047	0.049	1.0×	invariant
σ_η^2	4.21	0.15	27.6×	on ridge
A	1.54	—	—	on ridge
σ_ε^2	0.080	0.081	1.0×	invariant

Exactly the predicted ridge: with A free, μ , σ_η^2 and A wander together (σ_η^2 's sd is 27.6×

 wider than at $A=1$), while ϕ and σ_ε^2 are *invariant* (sd ratio 1.0), and the identified products stay tight ($A\mu$ median 1.73 vs truth 2.0; $A^2\sigma_\eta^2$ median 0.81 vs truth 1.0). The crucial point: *every* free- A parameter still *covers* its truth at 95%, yet the posterior is useless (σ_η^2 's CI is [0.02, 13]). **Identifiability is about posterior width, not coverage.** i.e., a non-identified model can cover the truth and still be uninformative.

Higher dimensions (A a matrix). The scalar c becomes a matrix: any invertible C commuting with Φ gives $\tilde{h}_t = C h_t$ another SVSSM ($\tilde{\mu} = C\mu$, $\tilde{\Sigma}_\eta = C\Sigma_\eta C^\top$) with $\hat{A} = AC^{-1}$ and identical y — an invariance group ranging from d_h parameters (diagonal Φ , distinct eigenvalues) to d_h^2 ($\Phi = \phi I$). Our V2 uses a *diagonal* Φ with distinct eigenvalues (no cross-asset persistence), so the relevant invariance is the d_h -parameter diagonal scale group. The difficulty now appears as a *measurable mixing collapse*. At $d_h=d_y=2$ ($\Phi = \text{diag}(0.9, 0.85)$), $T=200$, 4 chains $\times$ 3000 (full table in §7):

	A -entry bulk-ESS	max rank- $\hat{R}$	Vehtari pass
free A (12 par)	58–79 (of 12,000)	1.16	1/12
$A=I$ (8 par)	— (overall min 1958)	1.006	8/8

With A free, its entries get bulk-ESS 58–79 out of 12,000 draws ($\sim 0.6\%$ efficiency) at rank- $\hat{R} \geq 1.13$, this is a non-identifiability signature; only 1/12 marginals pass Vehtari. So the difficulty is real and *diagnosable*: it surfaces not as wide CIs alone but as catastrophic ESS on the unidentified directions. The remedy is restricting A §6; fixing $A=I$ leaves 8 parameters that all pass (min bulk-ESS 1958, rank- $\hat{R} \leq 1.006$).

6 Restrictions on A : what makes the model “meaningful”

Question. What restrictions on A make the model “meaningful”, and what does “meaningful” mean?

What “meaningful” means. Three criteria, increasing in strength: (i) *identifiable*: the parameters have a unique solution given (infinite) data, i.e. the ridge of §5 is removed; (ii) *interpretable*: the restriction carries a substantive reading, not an arbitrary pin; (iii) *matched*: the number of free parameters equals the number of quantities the data can inform (here 4: $A\mu$, $A^2\sigma_\eta^2$, ϕ , σ_ε).

The restriction: fix one scalar degree of freedom. The invariance of §5 is one-dimensional, so a *single* scalar constraint removes it. Any of $A=1$, $\sigma_\eta=1$, or $\mu=0$ works; the SV literature uniformly takes $\mathbf{A}=\mathbf{1}$, which is also the most interpretable; the observation then loads directly on the latent, so h_t is the log-volatility and (μ, ϕ, σ_η) keep their usual meaning.

The three restrictions are equivalent (demonstrated). On data from $(\mu, \phi, \sigma_\eta, A, \sigma_\varepsilon) = (2, 0.9, 0.4, 3, 0.3)$ (chosen so the three fixes land at *distinct* ridge points), each restriction recovers its own predicted ridge point and the *same* off-ridge invariants ($T=200$, 2 chains; every marginal covered):

restriction	param	predicted ridge	median (sd)	95% CI	covered
$A = 1$	μ	6.0	5.62 (0.54)	[4.50, 6.60]	✓
	ϕ	0.9	0.843 (0.048)	[0.750, 0.939]	✓
	σ_η^2	1.44	1.19 (0.20)	[0.82, 1.56]	✓
	σ_ε^2	0.09	0.099 (0.10)	[0.004, 0.37]	✓
$\sigma_\eta = 1$	μ	5.0	5.22 (0.73)	[3.94, 6.92]	✓
	ϕ	0.9	0.841 (0.047)	[0.752, 0.938]	✓
	A	1.2	1.094 (0.093)	[0.878, 1.242]	✓
	σ_ε^2	0.09	0.096 (0.099)	[0.004, 0.37]	✓
$\mu = 1$	ϕ	0.9	0.841 (0.048)	[0.751, 0.936]	✓
	σ_η^2	0.04	0.039 (0.011)	[0.023, 0.066]	✓
	A	6.0	5.60 (0.58)	[4.39, 6.64]	✓
	σ_ε^2	0.09	0.081 (0.092)	[0.002, 0.34]	✓

Each fix recovers *its* ridge point, and ϕ (≈ 0.84) and σ_ε^2 (≈ 0.09) agree across all three, they are the off-ridge invariants. So exactly one constraint is needed and any one suffices: the three are simply different coordinate choices for the same identified model. (One nuance: although all three *identify*, they are not equally well-*conditioned* for sampling, $A=1$ keeps the latent on a natural scale, whereas e.g. $\mu=0$ at a large true A can leave a poorly-scaled posterior.)

Multivariate. The invariance is the matrix group of §5, and its dimension depends on Φ . For *diagonal* Φ with *distinct eigenvalues*, the case we run, any C commuting with Φ is itself diagonal, so the only freedom is the d_h component *scales*; fixing the diagonal of A removes it, and $A=I$ is the simplest such fix. **This is the case we demonstrate:** the $A=I$ run of §5 (8 parameters, all passing Vehtari) confirms that one matrix-level restriction restores a clean, well-mixed posterior. Detailed table in the next section.

7 Which parameters need restricting for identifiability?

Question. Do we need to restrict *one* parameter or *all* of them for the model to be identifiable?

Principle. The number of parameters you must fix equals the dimension of the model’s *invariance group*, i.e., the set of parameter changes that leave the observed-data distribution unchanged. You fix exactly that many directions: not zero if a redundancy exists, and never *all* (that would leave nothing to estimate). For our two variants this number is 0 and 1 (scalar case), respectively. So the answer is “one, and only in V2”, never “all”.

V1 (canonical, $y_t = e^{h_t/2}\varepsilon_t$): no further restriction. The three dynamic parameters (μ, ϕ, σ_η) are jointly identifiable: §3 shows every marginal concentrating on truth as T grows ($\phi : 0.87 \rightarrow 0.92 \rightarrow 0.954$, $\sigma_\eta^2 \rightarrow 0.101$, μ covering $\approx 88\%$ of its mode-to-truth gap; $P(\phi < 0) : 13.6\% \rightarrow 4.9\%$). None of them needs to be restricted.

Weak vs. *non*-identifiability. V1 has no *strict* non-identifiability, but it is *weakly* identified at short T : a ϕ anti-persistence sub-mode, a wide μ marginal, and a (ϕ, σ_η) ridge that tightens with data (correlation $-0.24 \rightarrow -0.79$ from $T=50 \rightarrow 200$). These are remedied by *informative priors* ($\mu \sim \mathcal{N}(0, 1)$, $\phi_{\text{raw}} \sim \mathcal{N}(2, 0.5)$, $\log \sigma_\eta^2 \sim \mathcal{N}(-2, 1)$), which improve conditioning, and is not by restrictions. The distinction is the crux of the question: a *restriction* is mandatory for a genuinely redundant direction; a *prior* is an optional remedy for a poorly-informed but identified one.

V2 ($y_t = Ah_t + \sigma_\varepsilon \varepsilon_t$): exactly one restriction. Here the *additive* noise σ_ε becomes a genuine, separately *identifiable* parameter, the residual variance of y_t given the AR(1) structure, and is *never* restricted. Instead the loading A introduces a *new* scale invariance $(\mu, \sigma_\eta, A) \rightarrow (c\mu, c\sigma_\eta, A/c)$ (§5; the matrix group of §6 when $d > 1$), and exactly that many directions must be fixed, any one of $\{A, \mu, \sigma_\eta\}$ in the scalar case, all three equivalent (§6).

The cleanest evidence: non-identifiability is a Vehtari failure, and σ_ε passes it anyway ($d=2$). Restricting to a verdict per parameter, we run the multivariate V2 model ($d_y=d_h=2$, truth $\mu=(1, 1)$, $\Phi=\text{diag}(0.9, 0.85)$, $\Sigma_\eta=\text{diag}(0.25, 0.36)$, $A=\text{diag}(2, 2)$, $\Sigma_\varepsilon=0.09I$) two ways on the *same* $y_{1:200}$: **FREE** A (full 2×2 , 12-param) vs **FIXED** $A=I$ (8-param), each 4×3000 samples with Vehtari diagnostics (thresholds $\hat{R} \leq 1.01$, bulk-ESS ≥ 400 , tail-ESS ≥ 400).

Table 1: **FREE** A (12-param, 4×3000 samples). The four A entries (red) fail every threshold catastrophically (bulk-ESS ~ 70 of 12,000); σ_ε^2 (green) passes even here, because it sits *off* the identifiability ridge.

param	median	sd	$\hat{R}$	bulk-ESS	tail-ESS
A_{11}	−0.34	3.20	1.150	67	—
A_{12}	+0.03	2.92	1.126	74	—
A_{21}	−0.32	2.62	1.128	79	—
A_{22}	−0.67	3.10	1.159	58	—
μ_1	−0.09	0.85	1.077	82	281
μ_2	−0.07	0.85	1.047	91	280
ϕ_1	0.86	0.074	1.088	159	450
ϕ_2	0.83	0.082	1.086	134	275
$\sigma_{\eta,1}^2$	0.11	0.273	1.013	329	356
$\sigma_{\eta,2}^2$	0.10	0.432	1.048	249	—
$\sigma_{\varepsilon,1}^2$	0.065	0.069	1.007	611	701
$\sigma_{\varepsilon,2}^2$	0.106	0.112	1.005	379	605

Three things follow, and together they answer the question operationally. (i) **Non-identifiability is a measured diagnostic failure:** in FREE the A entries draw 58–79 effective samples from

Table 2: **FIXED** $A = I_2$ (8-param, 4×3000 samples). One matrix-level restriction makes *every* parameter pass every threshold ($\hat{R} \leq 1.006$, bulk-ESS ≥ 1958).

param	median	sd	$\hat{R}$	bulk-ESS	tail-ESS
μ_1	2.35	0.91	1.006	2043	2675
μ_2	1.69	0.45	1.000	9684	2081
ϕ_1	0.89	0.040	1.002	3925	2375
ϕ_2	0.81	0.055	1.003	1958	2852
$\sigma_{\eta,1}^2$	1.06	0.16	1.002	2888	4712
$\sigma_{\eta,2}^2$	1.08	0.21	1.002	2836	3820
$\sigma_{\varepsilon,1}^2$	0.076	0.076	1.001	3431	2246
$\sigma_{\varepsilon,2}^2$	0.125	0.120	1.002	2613	1572

12,000 ($\sim 0.5\%$ efficiency) with $\hat{R}$ up to 1.16, the 4 chains start slightly apart and explore *different segments of the ridge*, never agreeing. (ii) **The restriction is sufficient**: fixing $A=I$ lifts *every* parameter to bulk-ESS ≥ 1958 and $\hat{R} \leq 1.006$ (a $\sim 30\times$ jump), i.e., just one matrix-level restriction converts a non-mixing chain into a fully-mixed one. (iii) σ_ε **needs no restriction**: it passes Vehtari ($\hat{R}=1.007$, bulk-ESS 611) even *inside* the otherwise-broken 12-D FREE posterior, because the data pin the residual variance regardless of where the chain sits on the A -ridge. (The scalar three-way demo of §6 agrees: σ_ε^2 recovers ≈ 0.09 identically across all three fixes.) So in V2 you restrict *one* direction (the scale of A); ϕ and σ_ε stay free and identified.

Summary: The restriction count is the dimension of the invariance group:

variant	invariance	# restrict	which	always identified
V1 canonical	$\sigma_\varepsilon \leftrightarrow \mu$ level	0 further [†]	—	μ, ϕ, σ_η
V2 (scalar)	$(c\mu, c\sigma_\eta, A/c)$, 1-D	1	one of $\{A, \mu, \sigma_\eta\}$	ϕ, σ_ε
V2 (multivar.)	matrix group (§6)	dim(group)	$A=I$	ϕ, σ_ε

[†] one restriction is *implicit* in the canonical form ($\varepsilon_t \sim \mathcal{N}(0, 1)$, i.e. $\sigma_\varepsilon=1$); given it, no further restriction is needed.

Restricting *all* parameters is never correct: it would leave no free quantities to estimate, i.e. no statistical model. The right count is minimal that fix the redundant directions, estimate the rest. Over-restricting (e.g. fixing both A and σ_η in V2) is not wrong for identifiability but *wastes information*: it imposes a value the data could have supplied.

8 Identification: can different parameters give the same data?

Question. Can two different parameter vectors produce the same observed data, and if so, how do we restrict the model so the solution is unique?

Definition. A model is *identified* if the map $\theta \mapsto P_\theta(y_{1:T})$ is injective: distinct parameters imply distinct data laws. It fails when $\theta_1 \neq \theta_2$ are *observationally equivalent*, $P_{\theta_1} = P_{\theta_2}$, then no amount of data separates them, and the likelihood (hence the posterior, up to the prior) is constant along the set of equivalent θ .

Yes, in V2, along a one-parameter family. The cleanest “different parameters, same data” case is the V2 scale ridge. The observable law depends on $\theta = (\mu, \phi, \sigma_\eta, A, \sigma_\varepsilon)$ *only through the four invariants*

$$(A\mu, A^2\sigma_\eta^2, \phi, \sigma_\varepsilon),$$

since $\mathbb{E}[y_t] = A\mu$, the latent contributes $A^2\sigma_\eta^2/(1 - \phi^2)$ to $\text{Var}(y_t)$, ϕ sets the autocorrelation, and σ_ε the residual. The parameter is 5-dimensional but the data inform only 4 functions of it, so a *one-dimensional* set of θ shares every invariant and is observationally equivalent: $(\mu, \sigma_\eta, A) \rightarrow (c\mu, c\sigma_\eta, A/c)$ for any $c > 0$. §6 makes this concrete, the three vectors $(\mu=6, \sigma_\eta^2=1.44, A=1)$, $(\mu=5, \sigma_\eta^2=1, A=1.2)$, $(\mu=1, \sigma_\eta^2=0.04, A=6)$ fit the same data with identical ϕ and σ_ε , and the FREE- A chain of §7 literally *walks* this orbit (A entries sd ~ 3 , exploring different segments, never converging).

vs. weak identification (V1). A subtler, finite-sample cousin: the map is injective (identified), but nearly *flat* along a direction, so distinct θ give *nearly*, but not exactly, the same data. V1’s (ϕ, σ_η) ridge and ϕ sign sub-mode are of this kind: at small T two regions fit almost equally well, but the equivalence is broken as data accumulate. Crucially, the two regimes are *distinguishable by their posterior fingerprint*:

regime	example	posterior signature	remedy
exact non-ID	V2 (μ, σ_η, A) ridge	ridge; $\hat{R}$ fails at <i>all</i> T (1.13–1.16, §7)	restriction
weak ID	V1 (ϕ, σ_η) , ϕ -sign	broad but <i>concentrates</i> with T ($P(\phi < 0) : 13.6\% \rightarrow 4.9\%$)	informative prior

So “do different parameters give the same data?”: *exactly* yes in V2 (a restriction is mandatory), *approximately* yes in V1 at small T (cured by data, or sharpened by a prior). The diagnostic tells you which: a ridge whose $\hat{R}$ never falls is true non-identifiability; one that concentrates with T is merely weak information.

What the data *does* identify: *biased* stationary combinations (the V1 analogue of the V2 invariants). The weak ridge has a precise positive content. Just as V2’s data see only the invariants $(A\mu, A^2\sigma_\eta^2, \phi, \sigma_\varepsilon)$, the V1/multivariate data see the *stationary* quantities, i.e., the solution Σ_h of the Lyapunov equation $\Sigma_h = \Phi \Sigma_h \Phi^\top + \Sigma_\eta$, far better than the structural coordinates (Φ, Σ_η) that produce them. Deriving Σ_h from every draw of the $d=2$ sweep of §3 (truth in parentheses, these are the full- Φ Lyapunov stationary variances, so $\sigma_{h,0}^2=1.19 > 1$ reflects the ϕ_{off} cross-coupling *on top of* the unit per-component *diagonal* design of §3; coverage of truth at 95%):

T	derived stationary (data-identified)			structural $(\phi, \sigma \text{ ridge})$	
	$\sigma_{h,0}^2$ (1.19)	$\sigma_{h,1}^2$ (1.00)	ρ_h (0.15)	ϕ_{11} (0.80)	$\sigma_{\eta,1}^2$ (0.36)
50	1.63 (Y)	0.68 (Y)	−0.00 (Y)	0.84 (Y)	0.18 (Y)
200	2.39 (N)	1.28 (Y)	+0.04 (Y)	0.89 (Y)	0.25 (Y)
500	1.88 (N)	1.37 (Y)	+0.07 (Y)	0.92 (Y)	0.21 (Y)
1000	1.94 (N)	1.39 (N)	+0.14 (Y)	0.93 (N)	0.18 (Y)

Three points, all on converged chains ($\text{rank-}\hat{R} \leq 1.07$): (i) the cross-asset stationary correlation ρ_h is identified *precisely and accurately*, its median error collapses $-102\% \rightarrow -8\%$ as $T : 50 \rightarrow 1000$

(recovered 0.14 vs truth 0.15): a spillover invisible at short T but sharply pinned with data. (ii) ϕ_{11} (biased high) and $\sigma_{\eta,1}^2$ (biased low) move in *compensating* directions, so their combination $\sigma_h^2 = \sigma_\eta^2/(1 - \phi^2)$ is constrained far better than either coordinate, this is the classic SV ridge, now measured. (iii) But σ_h^2 inherits a structural *bias*, distinct from the ridge: it is *precise but biased*, which the next paragraph dissects.

The bias mechanism, and how simulation turns it into a correction. The bias originates in the HRS quasi-likelihood of §1. The exact observation is $\log y_t^2 = h_t + \log \varepsilon_t^2$, where $\log \varepsilon_t^2 \sim \log \chi_1^2$ is strongly left-skewed (mean -1.27 , variance 4.93). The filter replaces this density with a *single* Gaussian of matched first two moments, so the unmodelled skew and heavy left tail are absorbed as extra process noise, and the inferred innovation variance σ_η^2 is systematically *inflated*. Through $\sigma_h^2 = \sigma_\eta^2/(1 - \phi^2)$ that inflation passes straight into the stationary variance (and is compounded when ϕ is biased high). The key point is that this is a property of the *likelihood*, not of finite samples: it is a **fixed, deterministic offset**, not sampling noise.

The deviation from truth is therefore *bias, not variance*: the posterior median sits at a roughly constant offset from the truth, and as T grows the interval tightens *around that offset* rather than around the truth, $\sigma_{h,0}^2$'s 95% interval, wide enough to straddle the truth at $T=50$, has concentrated onto the biased value by $T=200$ ($\sigma_{h,1}^2$ by $T=1000$). This is not a defect to lament; it is precisely what makes the bias *usable*. Because it is deterministic and reproducible, demonstrated on converged chains (with `err_trigs` ≈ 0), so it lives in the likelihood and not the sampler, therefore, it can be *quantified in simulation*, where the truth is known. Running the identical filter+HMC pipeline on synthetic data across a grid of θ records the bias map $b : \theta_{\text{true}} \mapsto \hat{\theta}(\theta_{\text{true}})$ (the T -sweep table is exactly this measurement at one θ); on real data one then inverts it, reporting $\theta^{\text{corr}} = b^{-1}(\hat{\theta}_{\text{obs}})$; a sharp estimate corrected back onto the true parameter. This is the indirect-inference / simulation-based calibration logic: a precise, well-characterised bias is more valuable than imprecise unbiasedness, because precision is what lets the correction recover the true value. The same simulations that expose the bias are the ones that remove it from real-data estimates.

By contrast $\rho_h = \Sigma_h[0,1]/\sqrt{\Sigma_h[0,0]\Sigma_h[1,1]}$ is a *ratio*: the HRS inflation acts as an approximately common multiplicative scaling of the variance entries, which cancels in the normalisation. So ρ_h is **precise and accurate** (-8% error at $T=1000$) and needs no correction at all, while σ_h^2 is precise but biased. This is a clean illustration that which functionals survive a misspecified likelihood depends on whether the misspecification cancels within them.

9 Design of the neural operator

Question. What properties must the OT neural operator guarantee? Must it be differentiable w.r.t. the parameters, and smooth? Is ReLU enough — does a rectified-power unit (RePU, arXiv:2408.02697) help?

The operator and its three required properties. The operator replaces Sinkhorn: it maps the weighted source particles to equally weighted targets, conditioned on the model parameters θ and the per-step context. Concretely (DeepONet branch-trunk; §3 for the context vector),

$$T(x \mid \theta, c) = a(\theta, c) \odot x + \text{offset}(\theta, c) + \sum_{k=1}^K \underbrace{\text{splus}(\beta_k(\theta, c))}_{\text{branch}} \underbrace{\sigma(w_k^\top x + b_k)}_{\text{trunk}} w_k.$$

Three properties must hold, and each is guaranteed *by construction*:

property	why required	how guaranteed
valid OT map (monotone)	transport must be a Brenier map (∇ of a convex potential), else it is not OT	softplus-gated affine + ridge terms $\Rightarrow$ Jacobian $J_T = \text{diag}(a) + \sum_k \text{splus}(\beta_k) \sigma' w_k w_k^\top \succeq 0$ (mGradNet-M)
differentiable in θ	HMC needs $\nabla_\theta \log p$ by autodiff; the resample sits inside the recursion	θ enters the branch through smooth ops; autodiff returns exact $\partial T / \partial \theta$
smooth in the state x (C^1 , ideally C^2)	leapfrog needs a C^1 potential; the Monge–Ampère / $\log \det J$ loss needs C^2	C^∞ activations only (GELU branch, sigmoid trunk, soft-plus gates); no ReLU

Yes, it must be differentiable w.r.t. θ . The DPF exists to deliver $\nabla_\theta \log p(y_{1:T} \mid \theta)$ by autodiff so HMC can use it. The OT resample is one step of the filter recursion, and θ enters it through the operator’s context vector $(\mu, \phi, \log \sigma_\eta^2, \text{plus per-step } z_t, \text{ESS})$. For the gradient to be correct the transported particles must be a differentiable function of θ : Sinkhorn achieves this via entropic regularisation (a smooth, differentiable relaxation of the hard assignment), and a learned replacement must preserve it. If $\theta \mapsto T$ were non-differentiable — e.g. a hard arg max assignment, or a kink at the operating point — then $\nabla_\theta \log p$ would be wrong, zero, or discontinuous, and HMC’s leapfrog would integrate corrupted dynamics (biased trajectories, collapsed acceptance). This is also *why* θ must be an operator input (§3): an operator that did not take θ would have $\partial T / \partial \theta \equiv 0$ and would silently drop the resample step’s contribution to the gradient. We guarantee differentiability by composing only differentiable ops, so autodiff yields exact $\partial T / \partial \theta$.

Is ReLU differentiable? No. $\text{ReLU}(x) = \max(0, x)$ is C^0 but not C^1 : its derivative is the discontinuous Heaviside (a jump at 0) and its second derivative is a Dirac delta. Two failures follow for our use. (i) *Sampler*: the leapfrog integrator’s accuracy and near-symplecticity assume a C^1 (ideally C^2) potential; a gradient with jumps lowers acceptance and biases trajectories. (ii) *Training/geometry*: the $\log \det J$ Monge–Ampère loss and the PSD-Jacobian monotonicity guarantee need a well-defined second derivative, which ReLU lacks (0 almost everywhere plus deltas). So a piecewise-linear ReLU operator is doubly unsuitable.

Does RePU help? Partly, and we go further. arXiv:2408.02697 [7] (Kim & Kang, 2024) studies the rectified-power unit $\text{RePU}_p(x) = \max(0, x)^p$, the smooth generalisation of ReLU: $\text{RePU}_2 \in C^1$, $\text{RePU}_3 \in C^2$, so “something like this” *does* restore the differentiability ReLU lacks while keeping the rectified inductive bias, directly relevant to the smoothness our pipeline demands. The paper’s main message, however, is a caution: deep RePU networks are unstable to train (vanishing/exploding signal, a *criticality* failure of the RePU universality class), and they propose MRePU to recover stable training while keeping differentiability and universal approximation. *Our design sidesteps that trade-off entirely*: we use globally C^∞ activations (GELU, sigmoid, softplus), which have no kink *and* no RePU-style criticality pathology, and the softplus gates simultaneously deliver the PSD-Jacobian monotonicity for free. We thus obtain the differentiability and smoothness HMC requires without inheriting RePU’s instability, RePU/MRePU would be the route *if* one insisted on a rectified architecture, we didn’t find a need for it.

A coverage–capacity note. The operator’s fidelity is bounded by its θ -coverage (§3), but widening the training grid is *not* free at fixed capacity: spreading the $d=1$ ϕ grid $0.05 \rightarrow 0.20$ cost +23% sampling wall ($642 \rightarrow 791$ s/chain, a rougher target drives deeper NUTS trees) at *no* accuracy gain (§3). Size the grid to the posterior; widen the *network*, not just the grid, if broader coverage is required.

10 Training the operator: loss, the determinant, and JIT inside HMC

Question. How is the operator trained, what loss, and how do we know it has converged? The change-of-variables determinant is prohibitive in high dimension (its derivative more so), and getting the operator to jit-compile *inside* HMC is problematic. Why is it hard, and what do we do?

Loss and convergence. We train by *supervised* regression: minimise the mean-squared error between the operator’s transported particles and the Sinkhorn target map, over a grid of θ and filter states. This single choice is what defuses the determinant problem below — matching the map directly needs *no* determinant. Convergence is declared by plateau early-stopping on a held-out validation MSE with best-weight restore (e.g. at $d=1$, best val-MSE 0.0038 in tens of epochs / a few seconds), and is *validated downstream* by dropping the trained operator into the filter and checking that the log-likelihood matches Sinkhorn within Monte-Carlo noise ($|\Delta \log p| \leq 0.1$ at $d=1$). A pure Monge–Ampère (determinant-based) loss, by contrast, we found both unnecessary and *worse*: it minimises its own det-residual yet yields $|\Delta \log p| \sim 10^2$ versus Sinkhorn, because the determinant constraint alone underdetermines the map (it fixes volume change, not the translation).

The determinant problem, and why our design avoids it. The change-of-variables route needs $\log |\det J_T|$ of the transport: a general $d \times d$ determinant is $O(d^3)$ and its derivative (for backprop) costlier still, prohibitive per particle, per step, per gradient. The supervised loss above removes the determinant from training *entirely*.

Making it jit-compile inside HMC. Embedding a Keras operator in the filter’s `tf.while_loop` under `jit_compile=True` is genuinely fiddly, for three reasons we hit and fixed. (i) *No host syncs*: a single `.numpy()` on the operator path breaks the XLA cluster, removing three such calls turned an eager 3038 s HMC run into 327 s under XLA ($9.3\times$). (ii) *No stale state*: XLA folds `self.*` attributes into the trace as constants on the first call, so a θ stored on the model would be silently frozen at its first value while HMC keeps changing it; we thread θ and the context as tensor *arguments*, reserving `self.*` for true constants. (iii) *No retracing*: θ must be a `tf.Tensor` (not a Python float) or the function retraces every proposal (§2). With these, the operator is pure tensor ops inside the XLA graph giving one trace and fast gradient which is exactly what lets the NN-OT pipeline run under XLA at the speeds quoted in §3.

11 Accelerating HMC with a Hamiltonian neural network (LHNN)

Question. The filter makes each gradient evaluation expensive, yet HMC needs only the gradient of the log-posterior. Can a Hamiltonian neural network (Dhulipala et al., arXiv:2208.06120) that

learns H and ∇H replace the filter in the leapfrog inner loop and accelerate sampling? Does it apply here?

Why it fits our bottleneck. Each HMC gradient requires a full differentiable filter pass (LEDH flow + OT resample) — the dominant cost. L-HNN trains a small network on a short pilot of real-gradient trajectories to approximate the Hamiltonian $H_\theta(q, p)$ and returns ∇H_θ analytically, so the leapfrog steps no longer call the filter. We built it (`lhnn_nuts.py` + drivers, pilot/train weights cached for reuse), and it directly targets the right cost.

It works with large & measured speedups. A six-run $T=200$ benchmark (wide-shifted priors; all vs. the vanilla windowed-HMC reference at 39,842 s):

variant ($T=200$)	sampler / gradient	μ rank- $\hat{R}$	speedup	outcome
vanilla (ref)	fixed- $L=5$, dense M , real	1.008	$1\times$	PASS
L-HNN	fixed- $L=5$, id. M , surrogate	1.746	$5.97\times$	biased (basin-stuck)
L-HNN, disperse pilot	fixed- $L=5$, surrogate	1.850	$5.67\times$	worse (fix falsified)
L-HNN NUTS $d=5$	NUTS, surrogate	1.016	$9.13\times$	recovered (optimum)
L-HNN NUTS $d=7$	NUTS, surrogate	1.012	$4.75\times$	recovered

At moderate/narrow priors the savings are $\sim 95\%$ of real gradient evaluations; the wall optimum is NUTS depth-5 at $9.13\times$ with converged quality (rank- $\hat{R} \leq 1.016$). The cost is also T -favourable: surrogate-leapfrog cost is T -independent, so sampling grows *sublinearly* ($5.9 \rightarrow 10.3$ ks/chain over $T : 50 \rightarrow 1000$), repaying the ~ 67 -min pilot+train by $T \approx 200$ and netting $\sim 3\text{--}4\times$ at high T , exactly where the exact filter hurts most.

But not drop-in: the surrogate has a region of validity. At wide-shifted priors with long chains the posterior is tight and places mass in tails the pilot never visited; there the surrogate interpolates into *fake potential basins*, and fixed- L chains roll into per-init basins and stick, a *biased* posterior (μ rank- $\hat{R}$ 1.75 vs vanilla 1.008; 4 chains on 4 different μ basins). The comfort-zone contrast is sharp: the same model at a moderate prior gives `err_trigs` = 0, ~ 2 real-grads/iter, and recovers all three parameters; the wide prior gives `err_trigs` = 81, ~ 8.5 real-grads/iter, speedup gone. The boundary is the surrogate’s training coverage, not the dimension.

What fixes the bias — and what does not.

- **Broader pilot** (the obvious fix): *falsified* — a disperse 8-anchor pilot made it *worse* ($\hat{R}$ 1.85), because the same-capacity network spread thin creates *more* fake basins between training regions.
- **More capacity** ($128 \rightarrow 256\times 4$): no improvement (and 46% slower) at $d=2$, rules out surrogate under-fit as the cause.
- **Sampler architecture**: *works*. NUTS (multinomial sampling within the tree + Conrad-style final-state MH against the *real* target) recovers ($\hat{R}$ 1.025); a kinetic-energy MH correction (proper $-H_{\text{prop}} + H_{\text{init}}$) restores dispersion. The fix is sampler-level proposal correction, not trajectory monitoring.

This yields a clean rule: **fixed- L HMC** when you can precondition the dynamics (real gradient + dense mass); **NUTS** when you cannot (surrogate gradient, no preconditioner) — and it explains every row of the table.

Dimension ceiling. The fully accelerated pipeline with L-HNN on the gradient, NN-OT on the filter, fully recovered at $d=2$ (7/7 coverage, $\hat{R}$ 1.045, `err_trigs` ≈ 0). But $d=4$ ($T=200$, 18-D parameter) *failed* (killed at 6 h): the 128×3 surrogate underfits, all chains froze ($\alpha \rightarrow 0$), `err_trigs` ~ 300 /chain, the surrogate generalisation boundary triggered by dimension. The bind: windowed HMC pays a geometry cost, L-HNN hits the surrogate ceiling, and exact NUTS is compute-infeasible ($2.4 \text{ s/grad} \times 256/\text{iter}$).

Verdict. Yes, the method applies and is a real $4\text{--}9\times$ accelerator, but it is *not* drop-in. It requires (i) a sampler that does not trust the surrogate gradient blindly (NUTS + real-target MH + kinetic-MH), (ii) a moderate / in-distribution posterior, and (iii) $d \lesssim 2\text{--}3$ at current capacity. Operating rule: exact-gradient windowed HMC for wide-prior recovery; L-HNN for moderate-prior, high- T , or repeated-inference settings where the pilot cost amortises. The paper’s error-monitor safety story is empirically incomplete for our problem class (smooth fake basins where H_θ stays consistent but its gradient does not).

References

- [1] A. Vehtari, A. Gelman, D. Simpson, B. Carpenter and P.-C. Bürkner, *Rank-normalization, folding, and localization: An improved R -hat for assessing convergence of MCMC*, Bayesian Analysis **16** (2021) no.2, 667–718 [[arXiv:1903.08008](#)].
- [2] A. Harvey, E. Ruiz and N. Shephard, *Multivariate Stochastic Variance Models*, Rev. Econ. Stud. **61** (1994) no.2, 247–264 [DOI: 10.2307/2297980].
- [3] S. Kim, N. Shephard and S. Chib, *Stochastic Volatility: Likelihood Inference and Comparison with ARCH Models*, Rev. Econ. Stud. **65** (1998) no.3, 361–393 [DOI:10.1111/1467-937X.00050].
- [4] A. Corenflos, J. Thornton, G. Deligiannidis and A. Doucet, *Differentiable Particle Filtering via Entropy-Regularized Optimal Transport*, in *Proceedings of the 38th International Conference on Machine Learning (ICML 2021)*, PMLR **139** (2021) 2100–2111 [[arXiv:2102.07850](#)].
- [5] S. L. N. Dhulipala, Y. Che and M. D. Shields, *Efficient Bayesian Inference with Latent Hamiltonian Neural Networks in No-U-Turn Sampling*, J. Comput. Phys. **492** (2023) 112425 [DOI:10.1016/j.jcp.2023.112425].
- [6] L. Lu, P. Jin, G. Pang, Z. Zhang and G. E. Karniadakis, *Learning Nonlinear Operators via DeepONet Based on the Universal Approximation Theorem of Operators*, Nat. Mach. Intell. **3** (2021), 218–229 [[arXiv:1910.03193](#)].
- [7] T. Kim and M. Kang, *Why Rectified Power Unit Networks Fail and How to Improve It: An Effective Field Theory Perspective*, [[arXiv:2408.02697](#)].